

Practice Problems — Week 2: Basic Python Syntax

Rules: Attempt every problem on your own before checking the Solutions file. Solutions are in `Solutions.md`. Do not look ahead — working through the struggle is how learning happens.

Problem 1 — Temperature Converter

Objective

Write a script that converts a temperature from Celsius to Fahrenheit using the formula:

$$F = (C \times 9/5) + 32$$

Use a variable to store the Celsius value. Print both the Celsius and Fahrenheit values, clearly labelled.

Starter Code

```
celsius = 100
# YOUR CODE HERE
```

Expected Output

```
100 degrees Celsius = 212.0 degrees Fahrenheit
```

Hints

1. Calculate `fahrenheit` in one line using the formula. Python handles operator precedence correctly, but you may add parentheses for clarity.
2. `print()` will not concatenate a number and a string automatically — you will need to use `str()` to convert the numbers, or use a comma between arguments.
3. The result should be `212.0` (a float), not `212` (an int). Think about which operators produce floats.

Common Mistakes to Avoid

- Writing `9/5` and expecting an integer result — in Python 3, `/` always returns a float, so `9/5` is `1.8`. This is actually correct behaviour here.
- Forgetting to convert `celsius` and `fahrenheit` to strings when using `+` for concatenation.
- Incorrect formula order — double-check that multiplication happens before the `+` `32`.

Challenge Extension

Extend your script to also convert the result **back** from Fahrenheit to Celsius using the reverse

formula $C = (F - 32) \times 5/9$ and verify you get back to 100.0 .

Problem 2 — String Analysis

Objective

Given the string below, write code that performs four separate operations on it.

Starter Code

```
sentence = "automating the world with python"
# (a) print it in uppercase
# (b) print its length
# (c) print only the first word
# (d) print it reversed using slicing [::-1]
```

Hints

1. **(a)** — There is a built-in string method that converts all characters to uppercase.
2. **(b)** — There is a built-in function that counts the number of characters in a string.
3. **(c)** — There is a string method that splits a sentence into a list of words; the first element of the list is at index `[0]`.
4. **(d)** — Slicing with `[::-1]` reverses a sequence. Apply it to the full `sentence` string.

Common Mistakes to Avoid

- For part (c), do not manually count characters or use indexing to find the first word — use the appropriate string method.
- For part (d), `[::-1]` reverses the **entire string character by character** — spaces and all. The result will look like reversed text, not reversed words.
- String methods return a new string. If you call `sentence.upper()` without printing or storing the result, nothing will appear.

Challenge Extension

Add a fifth operation: **(e)** count how many times the letter "t" appears in the sentence. Look up the `count()` string method.

Problem 3 — Paycheck Calculator

Objective

Create a pay stub calculator. Define variables for an employee's information, compute their pay, and print a formatted summary.

Requirements

- Create variables for:
 - Employee name (`str`)
 - Hourly rate (`float`)
 - Hours worked (`int`)
- Calculate:
 - Gross pay = hourly rate $\times$ hours worked
 - Tax = gross pay $\times$ 0.20 (20% flat rate)
 - Net pay = gross pay – tax
- Print a formatted pay stub showing: employee name, hourly rate, hours worked, gross pay, tax deducted, and net pay.

Sample Variable Values

```
employee_name = "John Smith"  
hourly_rate = 18.50  
hours_worked = 40
```

Hints

1. Multiplication in Python uses `*`. Percentages can be expressed as decimals (20% = 0.20).
2. Format your output with clear labels, e.g., "Gross Pay: \$740.0".
3. All calculated values will be floats. Use `str()` to include them in concatenated strings, or use a comma in `print()`.

Common Mistakes to Avoid

- Forgetting to multiply, not add: gross pay is `rate * hours`, not `rate + hours`.
- Tax is a **deduction**, not the net pay. Net pay = gross – tax.
- Watch out for type errors when building label + value output strings.

Challenge Extension

Modify your script so the tax rate is stored in its own variable (e.g., `tax_rate = 0.20`) instead of hardcoded. Then add a second tax bracket: if gross pay exceeds \$800, apply a 25% tax rate instead.

Problem 4 — Comparison Operators

Objective

Create two variables and print the result of every comparison operator between them, with each output clearly labelled.

Starter Code

```
x = 15
y = 27
# Print the result of:
# equal to
# not equal to
# greater than
# less than
# greater than or equal to
# less than or equal to
```

Expected Output Format

```
x == y : False
x != y : True
x > y : False
x < y : True
x >= y : False
x <= y : True
```

Hints

1. Each comparison expression evaluates to either `True` or `False` — a `bool`.
2. You can print a label and a result using either string concatenation (convert the `bool` with `str()`) or a comma in `print()`.
3. There are exactly **six** comparison operators in Python. Make sure you include all of them.

Comparison Operators Reference

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to

Common Mistakes to Avoid

- Using `=` instead of `==` inside a comparison — `=` is assignment and will cause a `SyntaxError`.
- Confusing `>=` ("greater than OR equal to") with `>` ("strictly greater than").
- Forgetting that comparison results are `bool` values — you can store them in variables and use them in conditions.

Challenge Extension

Add three more comparisons: check if `x` is between `10` and `20` (inclusive) using `and`, check if either `x` or `y` is greater than `25` using `or`, and check if `x` is **not** equal to `15` using `not`.

Cross References

- **Concept files:** [Lab_2_1_Variables_and_DataTypes.md], [Lab_2_2_Operators.md], [Lab_2_3_StringOps_TypeConversion.md]
- **Solutions:** [Solutions.md]
- **Challenge:** [Challenge_Lab.md]
- **Quick reference:** [Revision_Sheet.md]